Decision Trees in Action

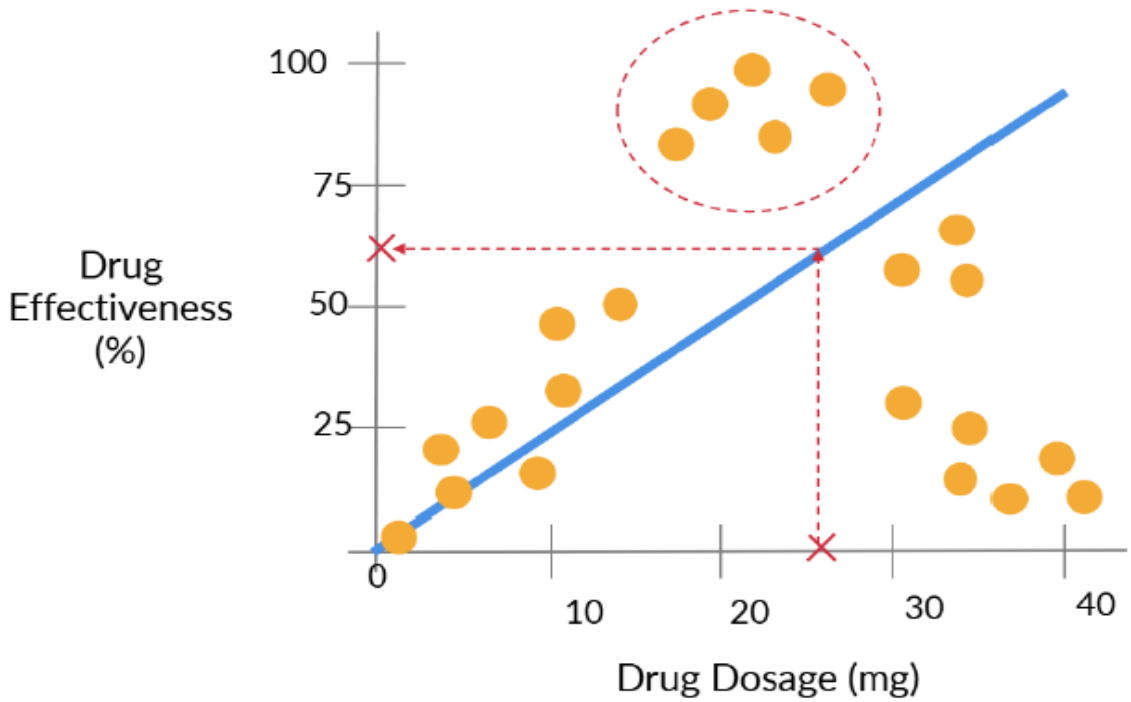
Dr. Vikash Kumar Sharma

Agenda

- Introduction
- Requirement of Decision Tree
- Structure and Types of Decision Tree
- Decision Tree Built
- Practical Decision Tree Model

Requirement of Decision Trees

Assumption: $Y = f(x) = \alpha + \sum \beta_i x_i$



Structure of Decision Trees

- A non-parametric supervised learning method
- Rule based approach to categorize or predict data based on homogeneity
- A decision tree is organized as a hierarchy of nodes connected by branches
- The origin of the tree is called the **root node**
- the data is split into subgroups using conditions on the features
- Each split forms new branches and creates **child nodes**
- This recursive process continues until the groups become simple enough to make a prediction
- The bottom nodes or the ends of a branch of a tree are called **leaf nodes**

Types of Decision Trees

- **Classification trees**: used when the target is categorical. Leaves of the tree assign class labels such as "*diseased*" or "*healthy*." Final prediction in leaf depends on majority.

- **Regression trees**: used when the target is numerical. Leaves predict values such as "*price of a house*" or "*number of years until an event*." Final prediction in leaf is avaerge value of target in the leaf.

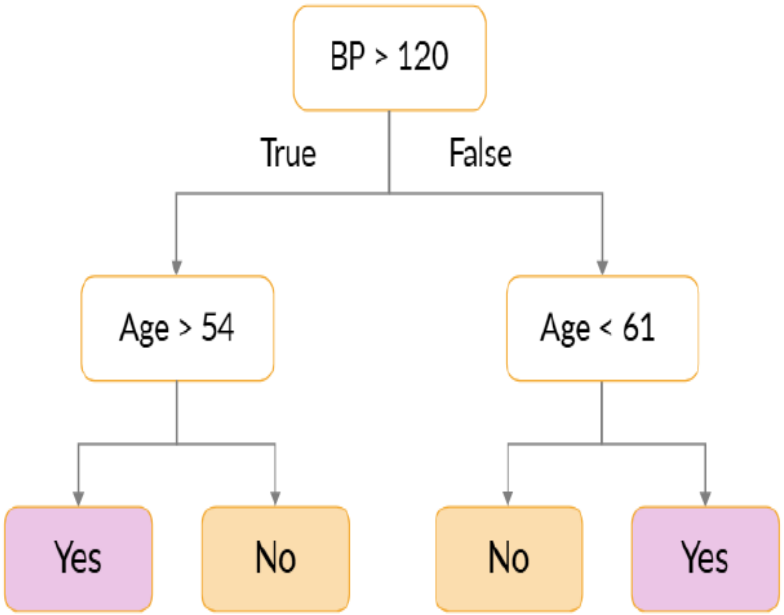
Splitting a Decision Tree

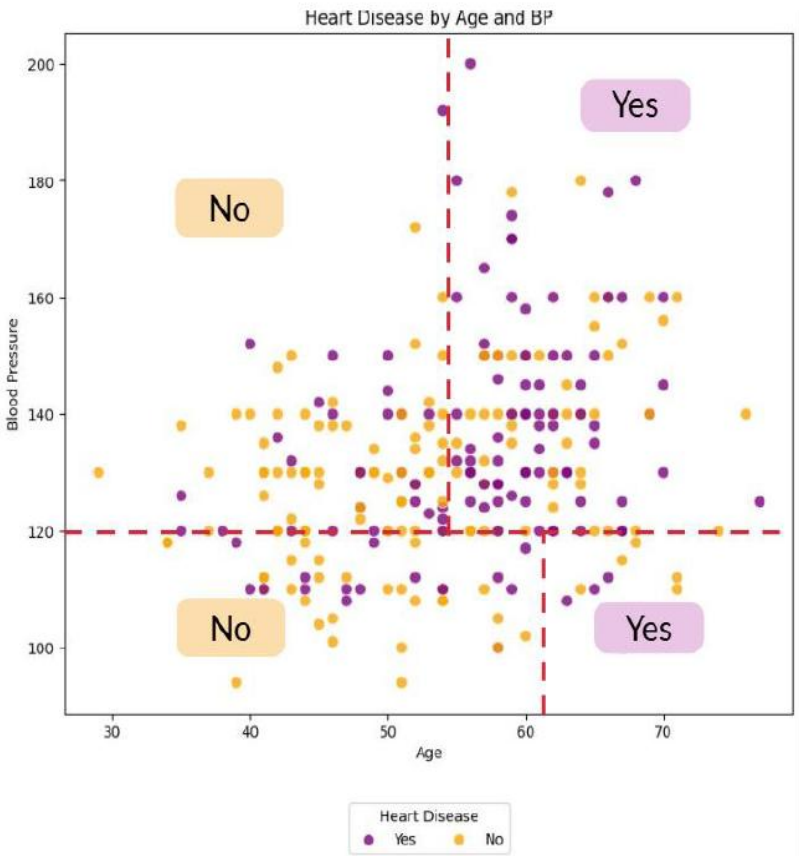
- At the start of training, a decision tree has the entire dataset sitting at the root. This is the first step, without any splits – the same label or average would be given to every data point.
- To improve on this, the tree begins by doing a check or test on a feature
- The usefulness of a decision rule lies in how well it separates the outcomes
- Once the first split is made, we move deeper and split the subgroups again using different rules
- At each node, the tree asks: what is the best feature and the best condition to divide this part of the data?

Splitting a Decision Tree

- Splitting on Categorical Features : the splitting process is straightforward. Each split tests whether a data point belongs to a subset of categories
- Splitting on Numerical Features: With numerical features, the tree must decide on a threshold. To do this, we first arrange the data points in order and then considers all possible splits in them. Each split occurs at midpoint of two data point. Then the next split occurs at the midpoint of the next set of points

Final Tree


BP > 120
True
False
Age > 54
Age < 61
Yes
No
No
Yes


Heart Disease by Age and BP
Yes
No
No
Yes
Blood Pressure
Age
200
180
160
140
120
100
30
40
50
60
70
Heart Disease
Yes
No

Best Split (Classification)

- Gini = $1 - \Sigma\, p^2$
- Lower Gini → Better split
- If p is close to 0 or 1, meaning that almost all points belong to the same class, the Gini index is near 0, showing high purity. If p is close to 0.5, the Gini index is higher, showing that the group is more mixed. A Gini index of 0.5 means equal proportions of both classes in the subnode.

Best Split (Classification)

- Entropy = $-\Sigma\ p \log_2(p)$
- $Information\ Gain = E_{parent} - E_{childern}$
- The split that gives the highest information gain is chosen, since it reduces uncertainty the most.

Best Split (Regression)

- Variance as the Measure of Impurity
- $Var = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \mu(y)\right)^2$
- $Variance\ Reduction = Var_{parent} - Var_{childern}$
- $Var_{children} = \frac{n_1}{n}Var_1 + \frac{n_2}{n}Var_2$
- The best split is the one that reduces the overall variance the most.

Training Of Decision Tree

- The algorithm repeatedly divides the dataset into smaller subsets, with each division chosen to maximise the improvement in prediction quality. E.g. CART
- The process begins with the entire dataset at the root
- The algorithm then considers all possible splits – across different features and for all data points in a single feature – and searches for the best possible split.
- Each candidate split is scored using an impurity measure: the Gini index or entropy for classification, and variance reduction for regression.

Hyperparameters

- **max_depth *(integer)*** – the maximum tree depth.
- **min_samples_split *(integer)*** – The minimum number of samples required to create a decision rule.
- **min_samples_leaf *(integer)*** – The minimum number of samples required to be in a leaf. A leaf will not be allowed to have a number of samples lower than this value. Ideal to overcome overfitting.
- **max_leaf_nodes *(integer)*** – The maximum number of leaves contained in the Decision Tree.
- **min_impurity_decrease *(integer)*** – The minimum impurity decrease value required to create a new decision rule.

Prediction on Test Data Set

- Majority in the node for classification (or the Probability)
- Mean of the target for regression in each node

Overfitting in Decision Trees

- An advantage of decision trees is their flexibility. They can capture complex patterns in the training set by repeatedly splitting. However, this same flexibility makes them prone to **overfitting**.

Stopping Criteria or Pre-Pruning

To stop overfitting, we need mechanisms to stop the growth earlier, before overfitting sets in. This strategy is called **pre-pruning** or **early stopping** or **truncation**. Instead of allowing the tree to grow to its full size, we impose rules that limit its depth or the number of points in its leaves. Mean of the target for regression in each node

Post Pruning

- *The Cost-Complexity Measure*

The formal way to balance accuracy with simplicity in trees is through the **cost-complexity function**: $R\alpha(T)=R(T)+\alpha\times|T|$

- where,
- $R(T)$ is the error of the tree on training data
- $|T|$ is the number of leaves
- $\alpha\geq 0$ is the complexity penalty

Cost Complexity Pruning (CCP)

- Grow the full tree until every node is pure or contains too few samples to split
- Generate a pruning sequence to iteratively remove branches that cause the smallest increase in the cost function. This creates a sequence of trees, from the largest down to the smallest
- Select the optimal tree with cross-validation to evaluate the sequence and choose the tree that minimises error on unseen data.

Error Against Tree Complexity

Feature Importance

- To compute feature importance, we track the total impurity reduction brought by each feature across all the splits in the tree

For a feature X_j, its importance is calculated as:

$$\text{Importance}(X_j) = \sum_{\text{splits on } X_j} \Delta \text{ Impurity}$$

where Δ Impurity is the decrease in Gini index, entropy, or variance produced by that split.

After summing these values over the whole tree, the importances are normalised so that they add up to 1 across all features.

Limitations of Feature Importance

- **Bias toward features with many levels:** A categorical variable with many categories has more opportunities to create splits and may appear artificially more important
- **Instability:** Decision trees are sensitive to small changes in data, so feature importance scores may vary from one dataset sample to another
- **Relative measure:** Importances are relative to the set of features in the model. If a strong predictor is removed, the importance of others may rise artificially
- **Bias towards training dataset:** Importance is calculated on training dataset

Strengths of Decision Trees

- Easy to understand and interpret; the rules can be followed step by step
- Naturally handle both numerical and categorical features
- No need for feature scaling or standardisation
- Capture non-linear relationships and interactions between features
- Robust to outliers, since splits depend only on orderings and groupings
- Can be visualised, making them useful for communication and explanation

Weaknesses of Decision Trees

- Highly prone to overfitting if not pruned
- Small changes in the data can lead to a very different tree (instability)
- Predictions are piecewise constant, which may be too crude for certain tasks
- Biased feature importance, especially toward variables with many categories
- Often less accurate than more advanced methods when used alone

Thank You

Appendix

Entropy Calculation (Step-by-Step)

- $p(Yes)=6/10$, $p(No)=4/10$
- Entropy $= -(0.6 \log_2 0.6 + 0.4 \log_2 0.4)$
- Entropy ≈ 0.97

Information Gain

- IG = Entropy(before) – Entropy(after)
- Higher IG = Better question

Gini Index (Quick Math)

- Gini = 1 – ($pYes^2$ + pNo^2)
- Pure node → Gini = 0

Pruning – Teaching Note

- Cut weak branches
- Reduce variance
- Improves test accuracy